

DNSX · SUPPLIED “LOYD EDITION” ARCHIVE

Security audit report

dnsx

Software security & audit-process review

Based on `ai_security_audit.md`. Source review, local reproductions, dependency advisory screening, and build/test validation.

Outcome: fixes required before relying on complete scan results.

Nine findings: six Medium and three Low, across security, supply-chain configuration, release integrity, and reliability. No Critical or High finding established in this scope.

What was established

- Actual DNS-over-HTTPS accepts an untrusted certificate; the version-check client has the same certificate-validation problem.
- The test helper executes shell syntax; impact requires less-trusted test input. A temporary-file recreate gap was reproduced under controlled timing.
- Functional comparisons accept incorrect same-length output. Empty-input and zero-worker runs can exit successfully; no-stdin stream mode crashes.

Important: passing audit reproductions mean a defect was demonstrated—not that the software is secure. Deployment and live CI behavior were not validated.

Audit observations: 08 October 2026, starting 16:11 UTC / 19:11 Africa/Cairo. Exact reproduction times appear on the finding pages. Introduction dates are not established.

Scope, identity & method

Input archive: dnsx (loyd edition).rar • 16,687,526 bytes • RAR v5

Archive SHA-256: dd876a7dc2ee7397edb7b5e8e8bc64ef66afec37db40f8ddf42f6ad9ea9d5ea4

Audit-file SHA-256: e42bab95e6cadaadbfd7f0e3f80cc464ac2535e1dd6a10178ec5246d3ab6257e

Bundled repository HEAD: 707bb16afa12a31e6783c4021a5d001a58e7a7a1

Bundled commit metadata: 2026-10-05T19:58:22+07:00 • build: untrack dnsx binary (#1082)

Observed software identity: Go module github.com/projectdiscovery/dnsx; banner version 1.3.1; go.mod requires Go 1.26.0.

The archive contains **116 entries, 82 regular files, and 54 non-Git project files**, including 21 Go files (tests included), nine workflows, manifests, scripts, docs, and images. No precompiled software executable was supplied. Bundled HEAD matches the working tree after ignoring line-ending whitespace. The archive label is not evidence of separate customization. Git metadata and remote URL do not authenticate upstream provenance.

Method applied from the supplied audit file

- Step 0: map CLI/library/test/CI entrypoints and trust boundaries; do not invent HTTP routes.
- Trace visible sources through validation to concrete sinks. Inspect pinned dependency source where application code delegates security controls.
- Quote exact file/line evidence, state preconditions, separate code confirmation from deployed exploitability, and challenge every candidate.
- Build the tool, run selected local tests and race detection, perform harmless reproductions, check module integrity, and query official advisories.
- Record limitations and rejected/downgraded claims. Proposed remediation was documented, not applied to the original software.

Compiler: checksum-verified Go 1.26.9. Scanner: govulncheck v1.8.0. Additional version-only screening: OSV querybatch for 105 declared module requirements. Full hashes, JSON findings, tests, and logs are in the evidence bundle.

02 / APPLICABILITY, NOT BLANKET PASS

Attack surface & module coverage

Entrypoint	Inputs / visible sinks / trust boundary
Production CLI: cmd/dnsx/main	Flags, local config, environment, files, stdin → DNS network calls, ASN/CDN helpers, stdout/output file, temp files and resume.cfg. No server auth boundary.
Public library: libs/dnsx	Caller options and hostnames → retryabledns transports, trace, AXFR, system resolver and CDN helper. Caller controls authority/scope.
Functional / integration harness	Testcase file, target, binary path and argv → bash -c. DNS listeners belong to test programs, not the production CLI.
CI / release workflows	PR/manual/tag/workflow_run/schedule triggers → hosted runners, remote actions, build/release credentials and Docker publication.
Audit module	Disposition
0: surface map; 1: sources/sinks	Completed for visible application/harness sources; delegated transports inspected. No application SQL, HTML/DOM or unsafe native deserialization sink found.
2, 4, 9: authorization/auth/tenancy	Not applicable to the supplied standalone CLI. PDCP client credential storage inspected; cloud server authorization and tenancy are outside the supplied code.
3: logic/state; 5: secrets/crypto	CLI lifecycle, input validation, output fidelity and TLS reviewed. Working-tree heuristic secret scan produced only secret references, no confirmed embedded secret.
6: dependencies/config; 7: outbound	OSV and govulncheck run; DoH/version-check TLS inspected. DNS/AXFR/trace networking is intended behavior, not automatically SSRF.
8: proxy/GraphQL/gRPC	Not applicable to visible architecture; no deployment proxy config supplied. No request-smuggling claim.
10: supply chain / CI/CD	Nine workflows, Dockerfile, release config, manifests and scripts reviewed. Remote action internals and org settings not audited.
11: webhooks/queues; 12: LLM agents	No production webhook consumer, broker or LLM/agent integration found. Not applicable, not certified safe.

Suggested risk order used: outbound transport trust → filesystem/input integrity → shell-based harness → CI/release provenance → output/error handling → dependency reachability.

F01 / MEDIUM / SECURITY DEFECT

DNS-over-HTTPS accepts untrusted certificates

Code confirmation: confirmed • confidence high. Security impact is subject to the preconditions below.

Observed: 2026-10-08T16:22:53.218074 UTC • introduction date not established

Mapping: CWE-295

Where: exact source evidence

retryabledns@v1.0.116/client.go : 82–85

```
httpClient := doh.NewHttpClient(  
    doh.WithTimeout(options.Timeout),  
    doh.WithInsecureSkipVerify(),
```

retryabledns@v1.0.116/doh/util.go : 31

```
transport.TLSClientConfig.InsecureSkipVerify = true
```

Why it matters & when it applies

An on-path party can impersonate the selected DoH service with an untrusted certificate and forge DNS results. This affects the encrypted DoH mode; UDP defaults are a separate, intentionally unencrypted mode.

Preconditions: DoH is explicitly configured; attacker can intercept traffic or impersonate its endpoint. No additional certificate pinning was found on this path.

Skeptical challenge & verification

CONFIRMED. Default Go HTTPS rejects the local self-signed server; actual DNSX accepted its 192.0.2.123 answer. DoT is a different transport and was not flagged.

Run `TestAuditDoHSelfSigned`. It uses `httptest.NewTLSServer` on loopback and `audit.test`; baseline HTTPS must reject the certificate, and the patched DNSX must reject it too.

Minimal remediation

Remove `doh.WithInsecureSkipVerify()` from `retryabledns.NewWithOptions`. Carry this as a reviewed dependency patch/fork or upgrade only to a release verified to restore certificate checks. Add a self-signed negative test.

Exact quotes, full traces and proposed diffs: `findings.json` / `proposed-fixes.txt`. Fixes not yet validated.

F02 / LOW / SECURITY DEFECT

Automatic version-check client disables TLS verification

Code confirmation: confirmed • confidence high. Security impact is subject to the preconditions below.

Observed: 2026-10-08T16:22:53.225165 UTC • introduction date not established

Mapping: CWE-295

Where: exact source evidence

internal/runner/options.go : 261–262

```
if !options.DisableUpdateCheck {  
    latestVersion, err := updateutils.GetToolVersionCallback("dnsx", version)()
```

utils@v0.11.6/update/update.go : 192–195

```
Transport: &http.Transport{  
    Proxy: http.ProxyFromEnvironment,  
    TLSClientConfig: &tls.Config{InsecureSkipVerify: true},  
},
```

Why it matters & when it applies

An on-path party can forge the reported latest version or observe transmitted machine/tool metadata. This is not evidence of replacement-binary execution: the release downloader uses a different path.

Preconditions: Automatic version check enabled (default), and attacker controls the network path or a configured interception proxy.

Skeptical challenge & verification

DOWNGRADED to Low. Self-signed acceptance reproduced. The insecure client is documented as version-check-only, so a claim of arbitrary malicious update installation through this client was rejected.

Run TestAuditVersionTLS against a loopback self-signed server. After changing the TLS configuration, this request must fail certificate verification. Use -duc as an immediate workaround.

Minimal remediation

Restore TLS certificate verification in utils/update.DefaultHttpClient. Immediate containment: use -duc. Do not assume this proves the separate updater is compromised.

Exact quotes, full traces and proposed diffs: findings.json / proposed-fixes.txt. Fixes not yet validated.

F03 / MEDIUM / CONDITIONAL TEST-HARNESS RISK

Test helpers turn target and arguments into shell code

Code confirmation: confirmed • confidence high. Security impact is subject to the preconditions below.

Observed: 2026-10-08T16:22:45.205815 UTC • introduction date not established

Mapping: CWE-78

Where: exact source evidence

internal/testutils/integration.go : 38–40

```
cmd := exec.Command("bash", "-c")
cmdLine := fmt.Sprintf(`echo %s | %s `, target, dnsxBinary)
cmdLine += strings.Join(args, " ")
```

Why it matters & when it applies

If this harness is fed less-trusted test data, a target containing shell syntax executes commands with the harness permissions. A harmless marker-file reproduction succeeded. The production DNS CLI does not call these test helpers.

Preconditions: A lower-trust party must control testcase data/arguments reaching the helper in a more-privileged harness. Committed trusted test literals alone do not establish an attacker boundary; ordinary PR builds already execute PR code.

Skeptical challenge & verification

CONFIRMED sink behavior, conditional security impact. Downgraded from production/remote RCE; no fork-secret leak or privileged CI exploitation was demonstrated.

Run `TestAuditShellExpansion`. It invokes a local stub, not the DNS tool, and creates only a temporary `AUDIT_OK` marker. After replacing the shell with `argv` execution, the marker must not appear.

Minimal remediation

Use `exec.Command(binary, argv...)` and `cmd.Stdin = strings.NewReader(target + "\n")` in both helpers. Pass `debug/silent` as literal `argv` entries, never shell text. Restrict who can choose the executable path.

Exact quotes, full traces and proposed diffs: `findings.json` / `proposed-fixes.txt`. Fixes not yet validated.

F04 / MEDIUM / CONDITIONAL LOCAL FILESYSTEM RISK

Stdin temporary-file creation loses atomicity and privacy

Code confirmation: confirmed • confidence high. Security impact is subject to the preconditions below.

Observed: 2026-10-08T16:22:53.225841 UTC • introduction date not established

Mapping: CWE-377 / CWE-367

Where: exact source evidence

utils@v0.11.6/file/file.go : 291, 299–300

```
tmpfile, err := os.CreateTemp("", "")
// ... closes the file ...
err = os.RemoveAll(tmpFileName)
return tmpFileName, err
```

internal/runner/runner.go : 306, 310, 312

```
tmpStdinFile, err := fileutil.GetTempFileName()
r.tmpStdinFile = tmpStdinFile
stdinFile, err := os.Create(r.tmpStdinFile)
```

Why it matters & when it applies

The create-delete-recreate gap allows a replacement symlink to redirect truncation and stdin writes where filesystem policy permits. `os.Create` also uses 0666 before umask, unlike `CreateTemp`'s 0600, making scan input world-readable under umask 022.

Preconditions: Non-stream piped stdin path is used. Reading requires access to the temp directory under a permissive umask; redirecting writes additionally requires winning the gap and passing OS symlink protections. Linux `protected_symlinks` can block cross-user attacks in sticky directories.

Skeptical challenge & verification

CONFIRMED unsafe sequence and controlled interleaving, not a measured natural race. Existing `CreateTemp` randomness was considered: deletion removes its atomic protection. Severity is conditional; same-user reproduction does not prove cross-user privilege escalation.

Run `TestAuditTempRace`: real helper returns a removed name; insert a symlink to a temporary victim, then reproduce `os.Create`. Keep the `CreateTemp` descriptor instead. Verify mode is 0600 and no reopen gap remains.

Minimal remediation

Use `os.CreateTemp("", "dnsx-stdin-")` directly in the runner, keep its descriptor, copy stdin through it, and remove its name on every return path. Do not call a create-delete filename helper.

Exact quotes, full traces and proposed diffs: `findings.json` / `proposed-fixes.txt`. Fixes not yet validated.

F05 / MEDIUM / SUPPLY-CHAIN CONFIGURATION RISK

Privileged CI actions are referenced by mutable tags

Code confirmation: confirmed • confidence high. Security impact is subject to the preconditions below.

Observed: 2026-10-08T16:26:27.938886 UTC • introduction date not established

Mapping: CWE-829

Where: exact source evidence

.github/workflows/auto-merge.yaml : 8–10, 17–18

```
permissions:
  contents: write
  pull-requests: write
# ...
- uses: projectdiscovery/actions/pr/approve@v1
- uses: projectdiscovery/actions/pr/merge@v1
```

Why it matters & when it applies

A moved or compromised action tag changes executed code without a reviewed workflow change. The auto-merge job explicitly has write permissions; release and Docker steps have sensitive credentials in their jobs. No action compromise was observed.

Preconditions: Attacker must control an action repository/tag or its publishing account, and the workflow must run. Actual token defaults and action behavior depend on repository settings and remote code not supplied.

Skeptical challenge & verification

CONFIRMED references and privilege grants, not confirmed compromise. workflow_run alone was not treated as a fork-code takeover; external approve/merge internals and branch rules remain unverified.

Resolve each action version to a reviewed full 40-character commit SHA, pin it, and use automated reviewed updates. Run CI on an owned test repository and verify only intended permissions are available.

Minimal remediation

Pin each action to a reviewed full commit SHA; no invented SHA is provided. Add default read-only permissions and narrow write scopes by job. Review remote action code and organization policies.

Exact quotes, full traces and proposed diffs: findings.json / proposed-fixes.txt. Fixes not yet validated.

F06 / MEDIUM / RELEASE-INTEGRITY DEFECT

Docker publication is not bound to a successful release commit

Code confirmation: confirmed • confidence high. Security impact is subject to the preconditions below.

Observed: 2026-10-08T16:26:27.938886 UTC • introduction date not established

Mapping: CWE-345 (artifact provenance)

Where: exact source evidence

.github/workflows/dockerhub-push.yml : 4–7, 14, 19

```
workflow_run:
  # workflow name omitted here; exact string in JSON
  types: [completed]
# ...
- uses: actions/checkout@v4
# Fetches /repos/projectdiscovery/dnsx/releases/latest separately.
```

Why it matters & when it applies

The workflow can publish on a failed release run, and the built commit is not constrained to the commit represented by its version tag. That can overwrite a versioned image with unrelated source, undermining reproducibility and audit traceability.

Preconditions: Workflow deployed as supplied; a release run completes or manual dispatch starts the Docker workflow. Real mismatch depends on timing/default branch and repository release state.

Skeptical challenge & verification

CONFIRMED missing success/ref binding in configuration; actual mislabelled image was not verified because GitHub/Docker deployment access was not provided. This is an integrity issue, not demonstrated attacker RCE.

In an owned test repository, simulate a failed release run and assert no push job runs. For success, assert image revision equals the triggering head_sha and tag metadata derives from that same immutable commit.

Minimal remediation

Gate workflow_run on conclusion == success. Checkout the triggering head_sha, fetch required tags, derive/validate the version tag from that same commit, and fail closed if the association cannot be established.

Exact quotes, full traces and proposed diffs: findings.json / proposed-fixes.txt. Fixes not yet validated.

F07 / LOW / AUDIT/TEST-PROCESS DEFECT

Functional comparisons can report success for wrong results

Code confirmation: confirmed • confidence high. Security impact is subject to the preconditions below.

Observed: 2026-10-08T16:22:45.346668 UTC • introduction date not established

Mapping: CWE-754 (quality control)

Where: exact source evidence

cmd/functional-test/main.go : 81–83

```
if len(mainOutput) == len(devOutput) {  
    return nil  
}
```

cmd/functional-test/run.sh : 20

```
go build -o dnsx$extension -v github.com/projectdiscovery/dnsx/cmd/dnsx
```

Why it matters & when it applies

A regression that produces incorrect results with the expected number of lines is accepted. In addition, the script labelled latest release builds the same local module path, not an independently selected release. This weakens confidence in existing functional-test passes.

Preconditions: Use of the provided functional harness. The same-checkout baseline follows normal Go module resolution from inside this module; no separate baseline download/ref is selected.

Skeptical challenge & verification

CONFIRMED. Two harmless stubs printing EXPECTED and INCORRECT each produced one line; runIndividualTestCase returned success. The script was inspected, not run against real public DNS targets.

Run TestAuditSameLengthMismatch; after fixing comparison it should reject the mismatch. Build the baseline in a separate worktree at a verified release commit and assert its revision differs when expected.

Minimal remediation

Compare result content, not just length: sort copies if ordering is intentionally nondeterministic, then slices.Equal. Build the baseline from a separate checkout at a verified release commit.

Exact quotes, full traces and proposed diffs: findings.json / proposed-fixes.txt. Fixes not yet validated.

F08 / MEDIUM / RELIABILITY AND AUDIT-INTEGRITY DEFECT

Incomplete scans and output failures can appear successful

Code confirmation: confirmed • confidence high. Security impact is subject to the preconditions below.

Observed: 2026-10-08T16:23:53.716804 UTC • introduction date not established

Mapping: CWE-252 / CWE-703

Where: exact source evidence

cmd/dnsx/dnsx.go : 38–40

```
// nolint:errcheck
dnsxRunner.Run()
dnsxRunner.Close()
```

internal/runner/runner.go : 623–629, 652

```
_ = w.Flush()
// ...
_, _ = w.WriteString(item + "\n")
// ...
for i := 0; i < r.options.Threads; i++ {
```

Why it matters & when it applies

Automation can receive exit status 0 for an empty-input failure or a run that resolves nothing. The actual output handler also returns normally when /dev/full rejects its deferred flush. That can cause incomplete or missing evidence to be mistaken for a completed audit.

Preconditions: No input, nonpositive -t, or an output filesystem error. The /dev/full reproduction ran only in the isolated test handler. A CLI /dev/full attempt did not obtain a DNS answer, so it was excluded as proof of output-loss behavior.

Skeptical challenge & verification

CONFIRMED via actual CLI for no input and -t 0, both exit 0/no output; confirmed output-error discard via handler test. Negative retries/timeouts are not separately claimed without dependency validation analysis.

Re-run recorded no_input and zero_threads checks; both should return nonzero with an actionable message after fixing. Run TestAuditOutputErrorDiscarded with explicit error propagation and expect a failure signal.

Minimal remediation

Check Run errors, reject Threads <= 0, and check WriteString/Flush/Close failures. Surface output failure in the process exit status; do not treat an empty result as proof that a scan completed.

Exact quotes, full traces and proposed diffs: findings.json / proposed-fixes.txt. Fixes not yet validated.

F09 / LOW / LOCAL AVAILABILITY DEFECT

Stream mode without stdin panics

Code confirmation: confirmed • confidence high. Security impact is subject to the preconditions below.

Observed: 2026-10-08T16:23:09 UTC • introduction date not established

Mapping: CWE-476

Where: exact source evidence

internal/runner/runner.go : 233, 243–247

```
var sc *bufio.Scanner
// ... file branch ...
} else if fileutil.HasStdin() {
    sc = bufio.NewScanner(os.Stdin)
}
for sc.Scan() {
```

Why it matters & when it applies

A routine invocation error crashes the process with a stack trace rather than a controlled input-validation error. This is not a remotely reachable service crash in the supplied architecture.

Preconditions: CLI is invoked with `-stream` and no piped input; normal CLI validation forbids a `hosts` flag in stream mode. A wrapper that allows this invocation can be disrupted.

Skeptical challenge & verification

CONFIRMED. `dnsx -duc -silent -nc -stream </dev/null` exited 2 with panic at `runner.go:247`. Treated as Low local availability impact, not remote DoS.

Run that CLI command in a disposable environment. Patched code should produce a concise nonzero error without a panic; normal piped stream input must still work.

Minimal remediation

Validate stream input before starting workers. If `sc` is `nil`, close the worker channel and report a controlled fatal/input error instead of invoking `sc.Scan`.

Exact quotes, full traces and proposed diffs: `findings.json` / `proposed-fixes.txt`. Fixes not yet validated.

13 / MATCHES ARE NOT AUTOMATICALLY EXPLOITS

Dependency advisory triage

Version-only OSV screening matched **five Go advisories in three modules**. govulncheck emitted module-level matches for all five and called-function traces only for goldmark. Its JSON-mode exit status 0 is not a clean bill of health. The matching advisories below come from scanner/API output, not guessed CVE descriptions.

Advisory / installed version	Reachability and contextual verdict	Upgrade / action
GO-2026-5841 klauspost/compress v1.18.2	s2.NewDict out-of-bounds read. Module match only; affected symbol not found reachable in scanner call graph. Exploit not established.	Fixed v1.18.7 per advisory.
GO-2026-5320 yuin/goldmark v1.7.4	renderImage/renderLink/renderAutoLink called through glamour during update release-note rendering. CLI renders terminal output; no browser HTML sink identified. Web XSS not established here.	Fixed v1.7.17. Keep as contextual dependency concern, not a confirmed CLI XSS finding.
GO-2026-5932 x/crypto v0.55.0	Unmaintained/unsafe OpenPGP package family. Module match only; no affected package/call path reported.	No fixed version listed; remove/replace OpenPGP if actually used. Do not claim upgrading resolves it.
GO-2026-6354 x/crypto v0.55.0	SSH undecided-channel deadlock DoS. Module match only; affected symbols not reported called.	Fixed v0.56.0.
GO-2026-6355 x/crypto v0.55.0	SSH established-channel deadlock DoS. Module match only; affected symbols not reported called.	Fixed v0.56.0.

What these results do not prove

Static reachability can miss dynamic behavior and does not evaluate every dependency logic flaw. F01/F02 were found by source inspection and local tests, independent of advisory matches. Toolchain vulnerabilities are evaluated against the Go 1.26.9 audit build, not an unknown original/deployed binary.

Sources: <https://api.osv.dev/v1/querybatch>; <https://vuln.go.dev>; individual records <https://api.osv.dev/v1/vulns/GO-2026-5320>, [GO-2026-5841](https://api.osv.dev/v1/vulns/GO-2026-5841), [GO-2026-5932](https://api.osv.dev/v1/vulns/GO-2026-5932), [GO-2026-6354](https://api.osv.dev/v1/vulns/GO-2026-6354) and [GO-2026-6355](https://api.osv.dev/v1/vulns/GO-2026-6355). Full records and call traces are retained.

14 / EVIDENCE, NOT REASSURANCE

Tests performed & actual outcomes

Check	Observed result / boundaries
Compilation and integrity	Go 1.26.9 build succeeded; go mod download succeeded; go mod verify: all modules verified. go vet ./... completed without reported issues.
Selected runner tests	24 top-level tests passed; 43 passed events including subtests. Not an additional independent 43-test suite.
Local runner suite with -race	25 top-level tests passed; 1 skipped. 44 passed events including subtests. No race reported in this run. The ASN expansion test was intentionally excluded.
Skipped existing test	TestRunner_InputWorkerStream skipped because the ASN API key was not configured. Separate CLI no-stdin failure was reproduced.
Audit security reproductions	Eight top-level reproductions passed across two runs. Passes validate the observed defect/behavior. Seven show/report defects; output-symlink behavior is contextual and not counted as a separate vulnerability.
CLI probes	No input and -t 0: exit 0, no results. -stream </dev/null: panic, exit 2 at runner.go:247. All used -duc; no update installed.
Output-failure probe distinction	CLI /dev/full attempt produced no DNS result and only a resolution warning: inconclusive for output loss. Isolated HandleOutput test supplied a record and confirmed ignored /dev/full flush failure.
Secret screening	Working-tree heuristic scan flagged five secret-related lines, all GitHub secret references rather than embedded secret values. No confirmed hardcoded secret; history and secret stores not comprehensively scanned.

Local reproductions

TLS tests used loopback self-signed HTTPS servers and a documentation-only 192.0.2.123 response. Shell test used a stub and a temporary marker. Temporary-file test controlled the replacement timing; it did not measure race-winning probability or bypass cross-user kernel protections. Existing wildcard tests used local DNS servers. No third-party exploitation, cloud metadata access, deployment, self-update installation, or production data modification was performed.

Reproduction output times: 16:22:45–16:24:16 UTC (19:22:45–19:24:16 Cairo). Race suite completed around 16:24:38 UTC. Exact machine-readable events appear in the bundle.

15 / EXPLICITLY UNVERIFIED

Audit-process problems & blind spots

Problem / limit	Effect on confidence and disposition
RAR extractor availability	Initial 7z command was missing; installed 7za could not read RAR v5. Resolved with local libarchive through ctypes, regular-file/path checks and extraction size limits. No software execution during extraction.
Initial tool availability	Go/compiler/scanner absent initially. Resolved by official checksum-verified Go 1.26.9 download and govulncheck v1.8.0 installation. Build, vet, tests and scanning then ran.
Incidental repository metadata change	Git status refreshed .git/index. Final hash check detected it; original bytes restored from archive. All 82 extracted regular files then matched their manifest hashes. No application fixes were applied.
No deployment / live CI access	Actual permissions, branch protection, action internals, image history, signing policy and network egress unknown. F05/F06 are code/config findings, not demonstrated CI takeover or published-image compromise.
No binary provenance	No original executable supplied; audit build differs from unknown deployed compiler/platform. Bundled Git origin/HEAD were not externally authenticated. No comprehensive malware/binary-reversing verdict.
Partial dynamic coverage	No broad public DNS scans, all integration/functional scripts, full fuzzing, cross-platform execution, cloud test environment or automated multi-user race campaign. Linux local reproductions only.
Secrets and history	No production credentials provided/needed. Full historical secret scanning, revocation checks and cloud secret-store review were not completed. No claim that secrets do not exist elsewhere.
Fix verification	Proposed fixes not applied to original software or validated as a complete repaired release. Re-audit patched dependency versions and CI configuration before closure.

Candidates rejected or downgraded

Rejected: SQL/IDOR/session/tenant/LLM findings without a corresponding architecture; intentional DNS, trace and AXFR traffic labelled as SSRF by default; actual secret leaks inferred from secret variable names; production RCE inferred from test helpers; binary-update compromise inferred from the version-check client; separate output-path vulnerability based solely on operator-selected symlinks.

The CheckInternalIPs flag marks internal answer data; it is not a network deny policy. If this CLI is later wrapped as a multi-user service, authorization, egress and per-user filesystem isolation require a separate audit.

16 / NEXT ACTIONS

Remediation order & evidence handoff

Priority	Recommended closure work
1 · Protect transport trust	F01: restore DoH verification and add negative certificate tests. F02: fix version-check TLS; use -duc until updated.
2 · Make results trustworthy	F08/F09: fail clearly on missing/invalid input and all output errors. Verify positive worker counts; retain successful end-to-end output tests.
3 · Protect local execution	F04: hold a 0600 CreateTemp descriptor. F03: remove shell string construction; distinguish trusted fixture code from untrusted testcase feeds.
4 · Repair validation pipeline	F07: compare content and use an independent verified release baseline. Test failures must be visible, not silently swallowed.
5 · Lock release provenance	F05: pin reviewed action SHAs and narrow permissions. F06: bind success, commit, tag and Docker image revision; test failure/manual paths.
6 · Upgrade and re-scan	Triage dependency upgrades with compatibility tests. Verify actual fixes for F01/F02 rather than assuming a version bump repairs them. Re-run govulncheck and local regression tests.

Evidence package

The accompanying ZIP contains findings.json with exact quotes/source-to-sink paths/preconditions, proposed-fixes.txt, archive file hashes, compiler/scanner metadata, test event logs, CLI results, advisory records, and the benign audit test sources. It deliberately excludes the software executable, original Git history, and production credentials.

Manifest hash checks verified that the supplied source files were preserved. Review diffs as proposed changes: some configuration actions require verified external SHAs/settings that are not invented in this report. No fixes have been represented as deployed or verified.

Acceptance criteria

- Self-signed DoH and metadata endpoints rejected; trusted endpoints continue to work.
- No shell interpretation of testcase targets; temp input remains 0600 with no create-delete-reopen gap.
- Missing input, invalid workers and output failure return clear failure signals; valid stream input completes without panic.
- Wrong same-count outputs fail comparisons; release and image metadata identify the same immutable commit.
- Repeat closure tests on the patched release and intended deployment.

Bottom line: this was a bounded, evidence-backed source and local-test audit. It found actionable problems but does not justify a claim of complete security or a clean production deployment.